Mighty Heroes

Architecture reference — namespace map, feature index, turn lifecycle

This document is the onboarding reference for any developer joining the Mighty Heroes codebase. Three sections, in order of usefulness:

1. Layer architecture. Which namespaces exist, what each is responsible for, and which layer is allowed to depend on which.

2. Feature-to-file map. For each gameplay question a developer might ask ("where does enemy health decrease?"), the file ownership chain.

3. Turn lifecycle timeline. A swim-lane diagram and step-by-step walkthrough of one full turn, end to end.

Conventions used throughout this document:

• **FSM** = Finite State Machine. **SO** = ScriptableObject. **HUD** = Heads-Up Display.

• File names are written as `ClassName.cs`.

• Namespace fragments are written as `Gameplay.Hero` (the `MightyHeroes` root prefix is implied).

1. Layer architecture

Five layers, each with a strict role. Arrows in the diagram below show the allowed direction of dependency — upper layers may reference lower ones, never the reverse. The composition root is the only place that knows about every other layer simultaneously.

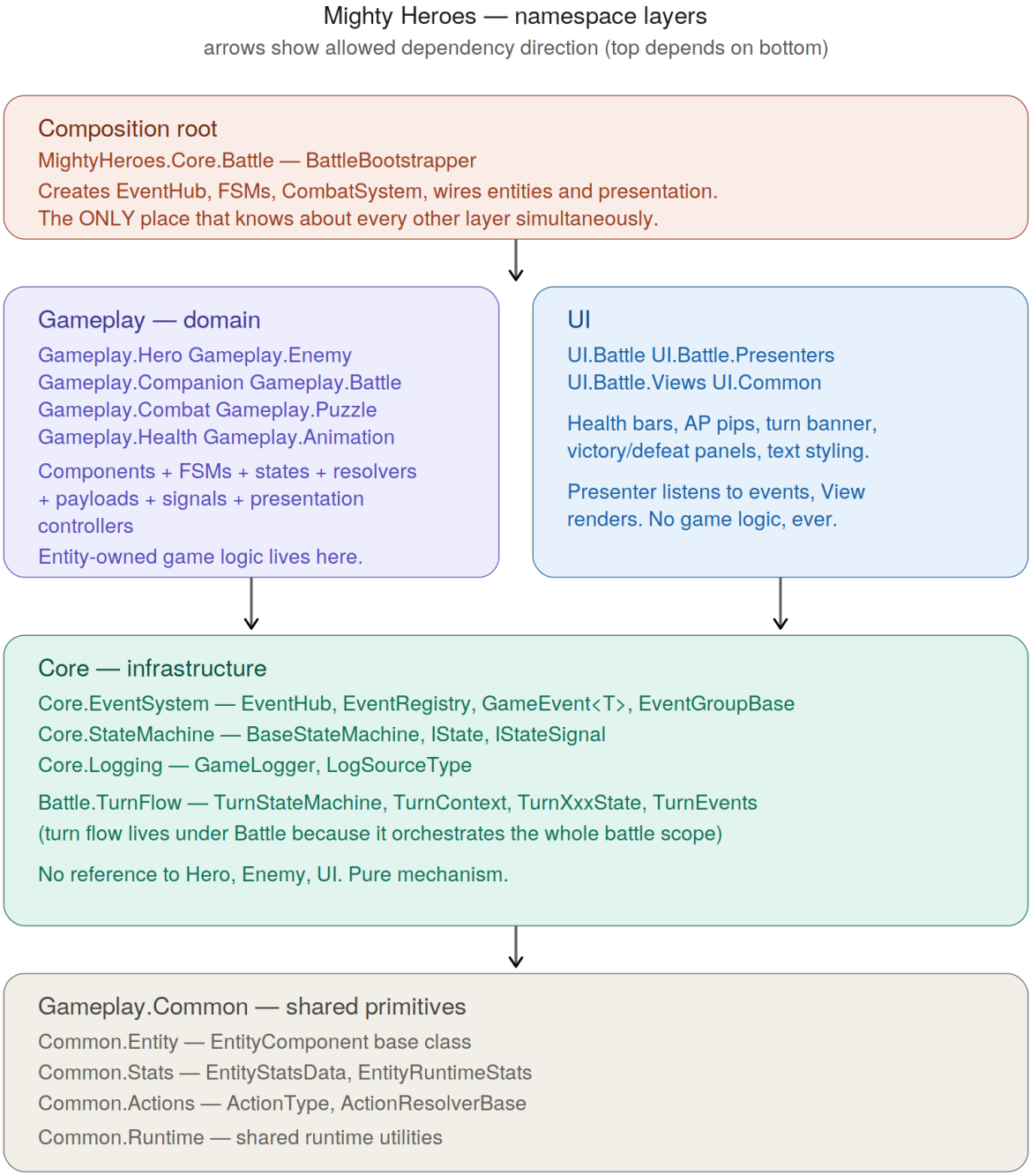


Figure 1 — Five-layer dependency map. Composition root at the top wires everything; shared primitives at the bottom underpin all layers.

Layer responsibilities

Layer	Owns	Forbidden
Composition root Core.Battle	Scene wiring, dependency injection, lifecycle (Dispose). Knows about every other layer.	Runtime gameplay decisions. Should never own state.
Gameplay Gameplay.* domain folders	Hero, Enemy, Companion, Combat, Health, Puzzle, Animation, Battle.Formation. FSMs, components, payloads, resolvers.	UI references. Composition root references.
UI UI.*	Presenters, views, HUD. Consumes gameplay events; never publishes its own. Pure presentation.	Game logic. Mutating entity state.
Core Core.EventSystem, Core.StateMachine, Core.Logging, Battle.TurnFlow	Pure mechanism. EventHub, base FSM, logger, turn flow orchestration.	Any reference to Hero, Enemy, UI, or specific gameplay concepts.
Gameplay.Common Common.Entity, Common.Stats, Common.Actions	Shared primitives: EntityComponent base, stats SO base, ActionResolverBase generic.	Knowledge of specific entity subclasses.

Namespace detail

Each namespace, the classes it owns, and what it is responsible for. Read the table the way a new developer would: look up the namespace, then find the file you need.

Core.* — pure infrastructure

Namespace	Classes	Responsibility
Core.EventSystem	EventHub, EventRegistry, GameEvent<T>, EventGroupBase, IEventHub, IEventRegistry	Type-safe in-memory pub/sub. Scoped per battle.
Core.StateMachine	BaseStateMachine, IState, IStateSignal, BaseState	Generic FSM. Used by Hero, Enemy, and Turn FSMs.
Core.StateMachine.Signals	IStateSignal marker interface	FSM-local async messaging. Different from events — signals stay inside one FSM.
Core.Logging	GameLogger, LogSourceType	Source-tagged logger filtered by LogSourceType.
Core.Battle	BattleBootstrapper	Scene composition root. Single MonoBehaviour, single instance per battle.

Known issue. `BattleIdGenerator.cs` lives in `MightyHeroes.Core.BattleSystem` instead of `MightyHeroes.Core.Battle`. Two names for the same concept — to be normalised in Sprint 1.

Battle.TurnFlow.* — turn orchestration

Turn flow lives under `Battle` rather than `Core` because it is not pure infrastructure — it owns battle-scope domain logic. The Turn FSM decides turn order but never executes a hero or enemy action itself; it raises an event and waits for the relevant FSM to respond.

Namespace	Classes	Responsibility
Battle.TurnFlow	TurnStateMachine, TurnContext, TurnSystemController	Owns turn flow lifecycle.
Battle.TurnFlow.States	TurnStartState, TurnHeroState, TurnCompanionState, TurnEnemyState, TurnEndState, TurnVictoryState, TurnDefeatedState	Each state raises an event or waits for a condition, then transitions.
Battle.TurnFlow.Events	TurnEvents group: TurnStartedEvent, TurnHeroPhaseStartedEvent, AllEnemiesDefeatedEvent, TurnBattleVictoryEvent, TurnBattleDefeatedEvent	Public broadcast of turn phase changes.
Battle.TurnFlow.Payloads	TurnStartedPayload, TurnHeroPhaseStartedPayload, etc.	Immutable data carriers.
Battle.TurnFlow.Types	TurnStateType, ITurnState	Discriminator enum and interface.

Gameplay.Common.* — shared primitives

Namespace	Classes	Responsibility
Common.Entity	EntityComponent (abstract)	Skeleton for all combat-capable entities. HP, AP, EntityId.
Common.Stats	EntityStatsData (abstract SO), EntityRuntimeStats	Static data + mutable runtime mirror.
Common.Actions	ActionType, ActionResolverBase<TIn, TOut>	Generic resolver base for puzzle-driven actions.
Common.Runtime	Shared utility classes	Cross-cutting runtime primitives.

Gameplay.Hero.* — hero domain

Namespace	Contains
Hero	HeroComponent (Unity adapter), HeroEvents, HeroAnimationEventProxy
Hero.StateMachine	HeroStateMachine, HeroBaseState
Hero.StateMachine.States	HeroIdleState, HeroPreReadyState, HeroReadyState, HeroAttackState, HeroRetreatState, HeroTakingHitState, HeroDefeatedState, HeroVictoryState
Hero.StateMachine.Signals	HeroAttackHitFrameSignal, HeroAttackClipEndedSignal, HeroAttackSequenceEndedSignal, HeroHitReactionEndedSignal, HeroRetreatEndedSignal, HeroVictorySequenceEndedSignal, HeroVictoryTurnEndedSignal, HeroCombatHitResolvedSignal, HeroTargetHitReactionEndedSignal
Hero.Actions	HeroActionResolver, HeroActionExecution
Hero.Stats	HeroStatsData (ScriptableObject)
Hero.Movement	HeroAttackMovementController (position mutation, retreat, root motion)
Hero.Presentation	HeroTargetFocusController (head/torso IK, focus blend)
Hero.Input	HeroTargetInputController (tap-to-target)
Hero.Payloads	HeroTurnEndedPayload, HeroAttackHitPayload, HeroAttackStartedPayload, HeroAttackSequenceEndedPayload, HeroVictoryTurnStartedPayload, HeroVictoryIdleStartedPayload, HeroHitReactionEndedPayload

Gameplay.Enemy.* — enemy domain

Mirrors the Hero namespace structure exactly: `Enemy.StateMachine`, `.States`, `.Signals`, `.AI`, `.Stats`, `.Presentation`, `.Payloads`. Differences: no input layer (AI decides), no retreat movement (fixed in formation), no puzzle subscription. Public API of EnemyComponent should be one-to-one with HeroComponent where applicable.

Gameplay.Combat.* — attack resolution

Namespace	Contains
Combat	CombatSystem — listens to HeroAttackHitEvent and EnemyAttackHitEvent, resolves damage, raises CombatDamageIntentEvent. Plain C# class, not a MonoBehaviour.
Combat.Interfaces	ICombatEntity, ICombatAttacker, ICombatTarget, ICombatResolver, IBattleField
Combat.Events	CombatEvents group
Combat.Payloads	CombatDamageIntentPayload, CombatHitResolvedPayload
Combat.Types	DamageType, HitDirection, SpecialAttackType
Combat.StateMachine.Base	ComboAttackState<TStateMachine> — generic base used by both HeroAttackState and EnemyAttackState

Gameplay.Health.* — defensive state

Namespace	Contains
Health	CombatHealth — plain C# model owning health + shield
Health.Events	HealthEvents group: DamageAppliedEvent, HealthChangedEvent, ShieldChangedEvent, EntityDiedEvent, EntityRevivedEvent, LifestealAppliedEvent
Health.Payloads	Payloads for each event above
Health.Types	HealthChangeReason

Gameplay.Battle.Formation — battlefield layout

BattleFormation, BattleLine, EnemyFormationConfig, EnemyFormationEntry, IFormation, IBattleField. Owns rows (front/mid/back), target selection, and enemy lookup. "Which enemy does the hero hit?" is answered here.

Gameplay.Puzzle.* — tile match module (third-party)

`Puzzle` (PuzzleController, third-party static singleton — to be wrapped with an adapter in Sprint 5), `Puzzle.Core` (GameEvents static, SingletonBehaviour base), `Puzzle.Grid` (GameGrid, GameGridElement, GameGridSpawning), `Puzzle.Events` (PuzzleEvents group — EventHub bridge), `Puzzle.Payloads` (PuzzleTilesMatchedPayload).

Gameplay.Animation.* — animation bridge

`Animation.Behaviours` — StateMachineBehaviours attached to animator states: AttackHitBehaviour, AttackClipEndedBehaviour, AttackSequenceStartedBehaviour, AttackSequenceEndedBehaviour, HitReactionEndedBehaviour, VictorySequenceEndedBehaviour. They forward animation events into the FSM via IAnimationSignalReceiver.

UI.* — presentation

`UI.Battle` (BattleUIPresenter, scene-level HUD), `UI.Battle.Presenters` (turn banner and phase presenters), `UI.Battle.Views` (HealthBarView, ShieldBarView, ActionPointView, TurnPhaseBannerView), `UI.Common` (TextStyleApplier, TextStyleConfig, TextStyleView).

2. Feature-to-file map

Each row is a question a developer might ask. The answer column shows the file ownership chain — read it as a path from the user's action to the system that responds.

Question	Answer
Where does enemy health actually decrease?	CombatHealth.ApplyDamage() — raised by EnemyComponent.OnDamageIntent() — raised by CombatSystem.OnHeroAttackHit(). The numeric formula itself lives in CombatResolver.ResolveDamage().
How do I make the 2nd hit of a combo deal base + bonus damage?	Edit CombatResolver.ResolveDamage() — it receives attackIntensity and you have access to the step index via the payload. The combo step counter is owned by ComboAttackState (in Combat.StateMachine.Base).
Where do I tune hero base stats (max HP, base attack, AP)?	ScriptableObject asset of type HeroStatsData (extends EntityStatsData). The asset is referenced by HeroComponent._heroStatsData in the inspector. Runtime mirror is EntityRuntimeStats.
Where is "puzzle match → hero action" decided?	HeroActionResolver.Resolve() — takes PuzzleTilesMatchedPayload, returns HeroActionExecution. Subscribed to in HeroReadyState.Enter(). The intensity formula sits in ActionResolverBase.ResolveIntensity().
How does the player select a different enemy target?	HeroTargetInputController reads input → BattleFormation.SelectTarget() updates state → raises CurrentTargetFocusChanged event → HeroTargetFocusController blends head/torso IK.
Who decides "all enemies dead, switch to victory"?	Currently BattleBootstrapper.OnEntityDied() — this is the antipattern flagged in Sprint 2 and will move to BattleFlowController. The deferred raise happens in OnHeroAttackSequenceEnded after the combo stack unwinds.
Where is the turn order enforced (Hero → Companion → Enemies)?	TurnStateMachine with sequential states: TurnStartState → TurnHeroState → TurnCompanionState → TurnEnemyState → TurnEndState. Each state raises a TurnXxxPhaseStartedEvent.
Where does enemy AI pick its action?	EnemyActionDecision + EnemyActionResolver.Resolve(). Called from EnemyAttackState when the enemy's turn begins. AI logic is currently minimal — extend EnemyActionResolver when adding behaviour trees or weighted decisions.
Where is the hit-frame raised by the attack animation?	Animator clip has an AttackHitBehaviour StateMachineBehaviour attached. It calls HeroAnimationEventProxy.NotifyAttackHit() → HeroComponent.NotifyAttackHit() → HeroStateMachine.NotifyAttackHitFrame() → active state.
Where does the hero physically retreat after attacking?	HeroAttackMovementController.StartRetreat() using DOTween. Triggered today by HeroComponent.NotifyAttackSequenceEnded() direct call. Sprint 3 will convert this to event subscription.
Where does the health bar HUD update?	HealthBarView is the MonoBehaviour view. It's refreshed from HeroComponent.RefreshHealthView() and EnemyComponent.RefreshHealthView() after every OnDamageIntent. Owned by the entity GameObject (world-space canvas with BillboardCanvas).

Question	Answer
Where do I add a new buff type?	Not implemented yet. Will live in Gameplay.Common.Stats as a runtime modifier on EntityRuntimeStats, with a dedicated BuffEvents group under Core.EventSystem.
Where do I add a new attack variant or class skill?	Animator: add new clip + behaviours. Resolver: extend HeroActionResolver.ResolveAttackVariant() (currently hardcoded to return 1). Data: add SerializedField on HeroStatsData for variant config.

3. Turn lifecycle timeline

Below is a swim-lane timeline of one full standard turn — hero attacks, then enemies retaliate. Five lanes, each representing a system that participates: Turn FSM, Hero FSM, Animator behaviours, EventHub, and the Combat / Health subsystems.

How to read it: solid arrows are synchronous calls (method call or signal forward); dashed leader lines are events raised on EventHub (publish — subscribers respond in parallel). Time flows top to bottom; each lane shows what its system is doing at each moment.

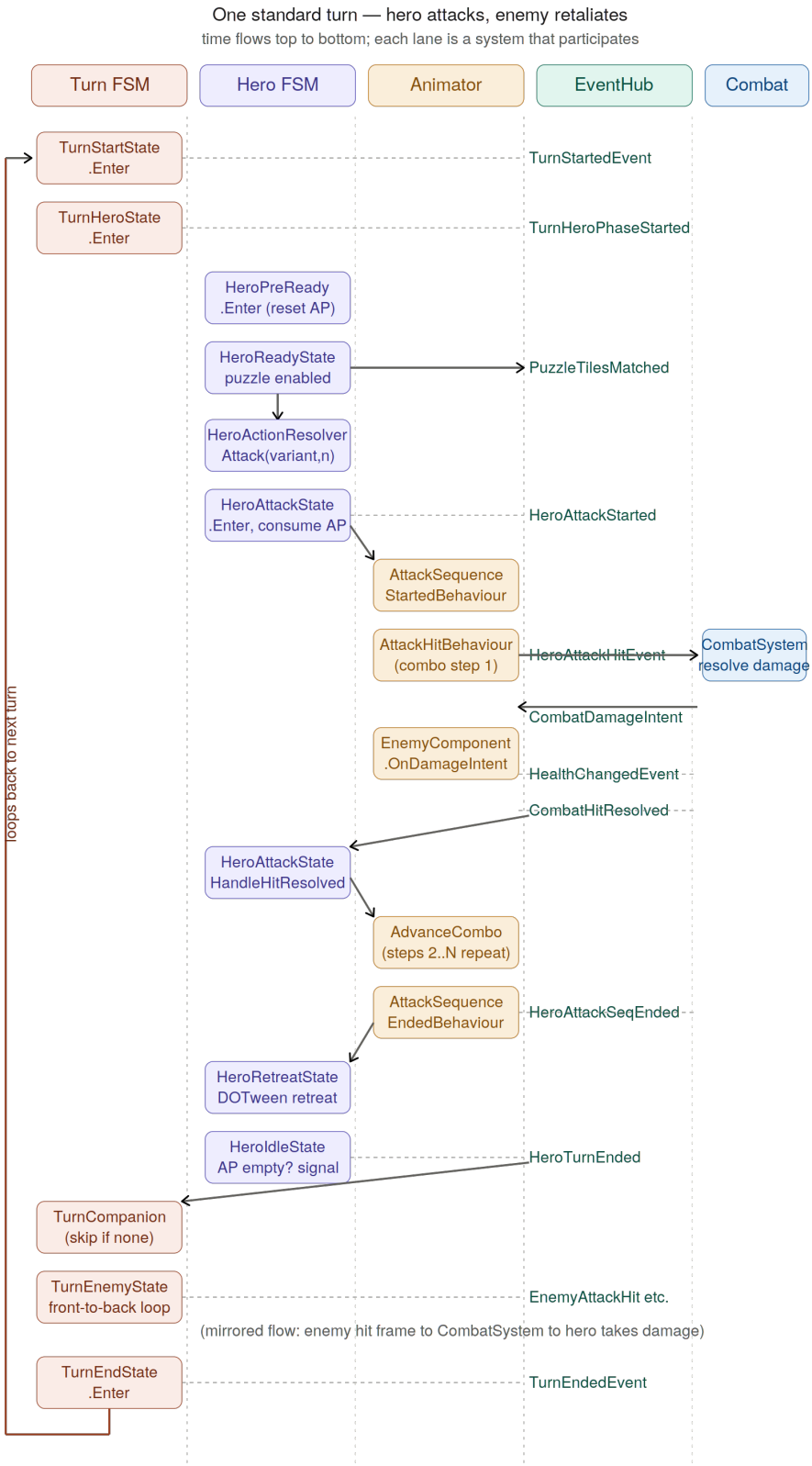


Figure 2 — Swim-lane timeline of one standard turn.

Step-by-step walkthrough

The same sequence in prose. Each item is a tick on the timeline.

Turn start. TurnStartState raises TurnStartedEvent. TurnHeroState raises TurnHeroPhaseStartedEvent. HeroStateMachine listens and transitions HeroIdleState → HeroPreReadyState.

Hero readies. HeroPreReadyState.Enter resets AP and activates the AP HUD. After a short pre-action delay, it switches to HeroReadyState which enables puzzle input and subscribes to PuzzleTilesMatchedEvent.

Player makes a match. PuzzleAdapter (planned) forwards the third-party puzzle resolution into PuzzleTilesMatchedEvent on EventHub. HeroReadyState receives this event.

Action resolved. HeroActionResolver.Resolve() takes the match payload and returns HeroActionExecution describing the attack variant and intensity (combo length). HeroReadyState switches to HeroAttackState.

Attack begins. HeroAttackState.Enter consumes one AP via HeroComponent.TryConsumeActionPoints(1) and raises HeroAttackStartedEvent. HeroComponent.PlayAttack triggers the animator.

Animator hit frame. At the authored hit frame, AttackHitBehaviour calls HeroAnimationEventProxy.NotifyAttackHit() → HeroComponent.NotifyAttackHit() → HeroStateMachine.NotifyAttackHitFrame() → active state. HeroAttackState.HandleHitFrame raises HeroAttackHitEvent.

Combat resolves damage. CombatSystem listens to HeroAttackHitEvent, asks IBattleField for the current target, asks CombatResolver to compute damage, raises CombatDamageIntentEvent.

Target takes damage. EnemyComponent.OnDamageIntent self-identifies by EntityId, calls Health.ApplyDamage. Health raises HealthChangedEvent, ShieldChangedEvent, and DamageAppliedEvent as appropriate. EnemyComponent raises CombatHitResolvedEvent.

Combo advances. HeroStateMachine listens to CombatHitResolvedEvent globally and forwards it into the active state as HeroCombatHitResolvedSignal. HeroAttackState.HandleCombatHitResolved calls HeroComponent.AdvanceCombo, which sets the AdvanceCombo animator trigger to play the next clip. Steps 6–9 repeat for each combo step.

Sequence ends. When the final clip finishes, AttackSequenceEndedBehaviour raises HeroAttackSequenceEndedEvent. HeroAttackState.OnSequenceEnded transitions to HeroRetreatState.

Hero retreats. HeroAttackMovementController.StartRetreat runs a DOTween tween back to the captured pre-attack position. When complete it raises RetreatCompleted, which HeroComponent forwards to FSM as HeroRetreatEndedSignal.

Hero turn ends. HeroRetreatState transitions to HeroIdleState. If hero AP is exhausted, HeroIdleState raises HeroTurnEndedEvent.

Companions and enemies act. TurnStateMachine advances to TurnCompanionState (skipped if no companions), then TurnEnemyState. Enemies act front-to-back in formation order. The hit-frame / damage / health flow is mirrored for each enemy.

Turn ends. TurnEndState raises TurnEndedEvent. The Turn FSM loops back to TurnStartState and a new turn begins.

Battle-ending edge cases

All enemies die mid-combo

EntityDiedEvent fires while HeroAttackState is still executing a combo. Today BattleBootstrapper.OnEntityDied (after Sprint 2 this moves to BattleFlowController) sets a deferred flag rather than switching turn state immediately — switching mid-combo would cause reentrant FSM transitions. When the combo's HeroAttackSequenceEndedEvent fires, the flag triggers AllEnemiesDefeatedEvent → TurnVictoryState → HeroVictoryState. This pattern was added after an early bug where mid-combo state transitions corrupted the turn FSM.

Hero dies

EntityDiedEvent fires for the hero. BattleFlowController calls TurnContext.EndBattle() and the Turn FSM switches to TurnDefeatedState immediately — no deferred flag needed, because the hero entering its terminal HeroDefeatedState halts the combo anyway. There is no re-entry path into hero active states after death, so no reentrancy risk.

Appendix — known issues and pending refactors

These are tracked in the architectural review and grouped into sprints. Listed here so a new developer immediately knows which patterns are deliberate and which are awaiting cleanup.

Sprint	Item
Sprint 1 cleanup	Remove dead code (HeroAttackState._victoryTurnIndex, HeroStateMachine._isVictoryPending, BattleBootstrapper.OnAllEnemiesDefeated, HeroAttackMovementController.SetLayerWeight body). Normalise namespaces: HeroCameraRig and WeaponType need the MightyHeroes prefix; BattleIdGenerator moves from Core.BattleSystem to Core.Battle. Rename files: DamagedAppliedPayload.cs to DamageAppliedPayload.cs; SingletonBeheviour.cs to SingletonBehaviour.cs. Replace raw Debug.Log with GameLogger. Implement or remove EnemyComponent.NotifyAttackSequenceStarted (currently throws).
Sprint 2 bootstrapper	Introduce IEventHubBound interface for auto-binding scene MonoBehaviours; remove per-component Initialize methods from BattleBootstrapper. Extract BattleFlowController to own death and battle-end logic instead of doing it in the bootstrapper.
Sprint 3 coupling	Convert HeroAttackMovementController to event-driven (subscribe to HeroAttackStartedEvent and HeroAttackSequenceEndedEvent instead of being called directly by HeroComponent). Same pattern for HeroTargetFocusController victory hooks. Replace `new` shadowing in EntityComponent AP methods with virtual/override.
Sprint 4 symmetry	Align HeroComponent and EnemyComponent public APIs one to one. Same method names and ordering for everything that maps; asymmetries kept only for genuinely entity-specific behaviour.
Sprint 5 puzzle	Add PuzzleAdapter MonoBehaviour. Remove all references to PuzzleController.Instance and GameEvents static events from outside the adapter — the third-party puzzle module becomes fully encapsulated.
Sprint 6 victory rotator	Decide on HeroVictoryRotator: either fold the 180-degree rotation into the victory animation's root motion and delete the component, or move it under HeroComponent and bind via IEventHubBound. Either way, remove the bootstrapper field.